ApacheCon

Apache Performance Tuning

Part 1: Scaling Up

Sander Temme <sander@temme.net>

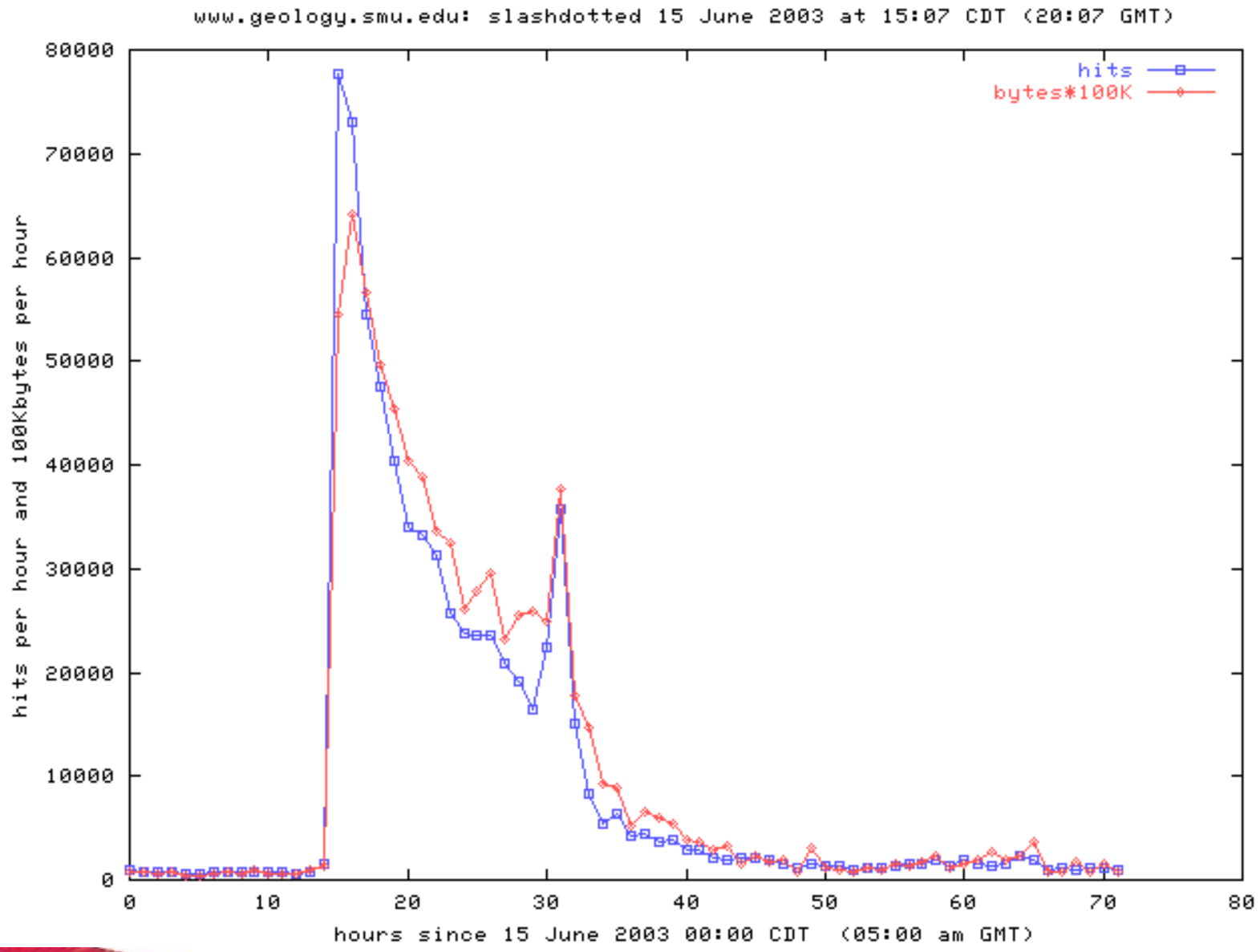


Leading the Wave
of Open Source

http://httpd.apache.org/docs/1.3/misc/perf-tuning.html
says:

"Apache is a general webserver, which is designed to be correct first, and fast second. Even so, its performance is quite satisfactory. Most sites have less than 10Mbits of outgoing bandwidth, which Apache can fill using only a low end Pentium-based webserver."




www.geology.smu.edu: slashdotted 15 June 2003 at 15:07 CDT (20:07 GMT)
hits
bytes*100K
hits per hour and 100Kbytes per hour
0
10000
20000
30000
40000
50000
60000
70000
80000
0
10
20
30
40
50
60
70
80
hours since 15 June 2003 00:00 CDT (05:00 am GMT)

The Silver Bullet

"Make my web server go fast, please"





Sorry

There is No Silver Bullet



No Silver Bullet

- Every site is different
  - Dynamic Content
  - Static Content
- Different Traffic Patterns
- SSL or Plaintext
- No Cookie Cutter Approach



Performance Tuning

- Ready
- Aim
- Fire
- Aim some more
- Understand
- Aim even more
- Fire



In practice

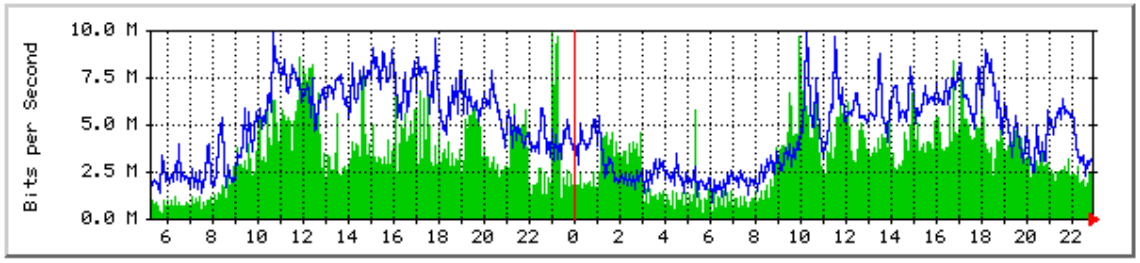
- Setup for analysis
- Monitor and analyse
- Tune configuration
- Lather, rinse, repeat



Monitoring Your Server

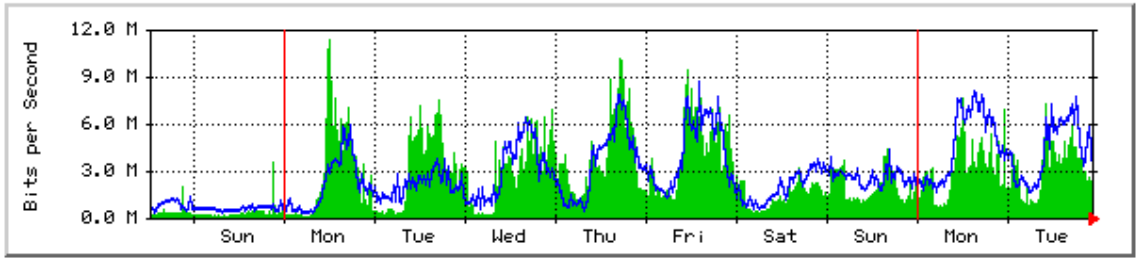
The statistics were last updated **Tuesday, 13 January 1998 at 23:00** ,
at which time **'Cgate1.cern.ch'** had been up for **25 days, 13:20:36**.

`Daily' Graph (5 Minute Average)



Max In: 9938.0 kb/s (9.9%) Average In: 3403.8 kb/s (3.4%) Current In: 1900.5 kb/s (1.9%)
Max Out: 9946.0 kb/s (9.9%) Average Out: 4820.8 kb/s (4.8%) Current Out: 3306.9 kb/s (3.3%)

`Weekly' Graph (30 Minute Average)



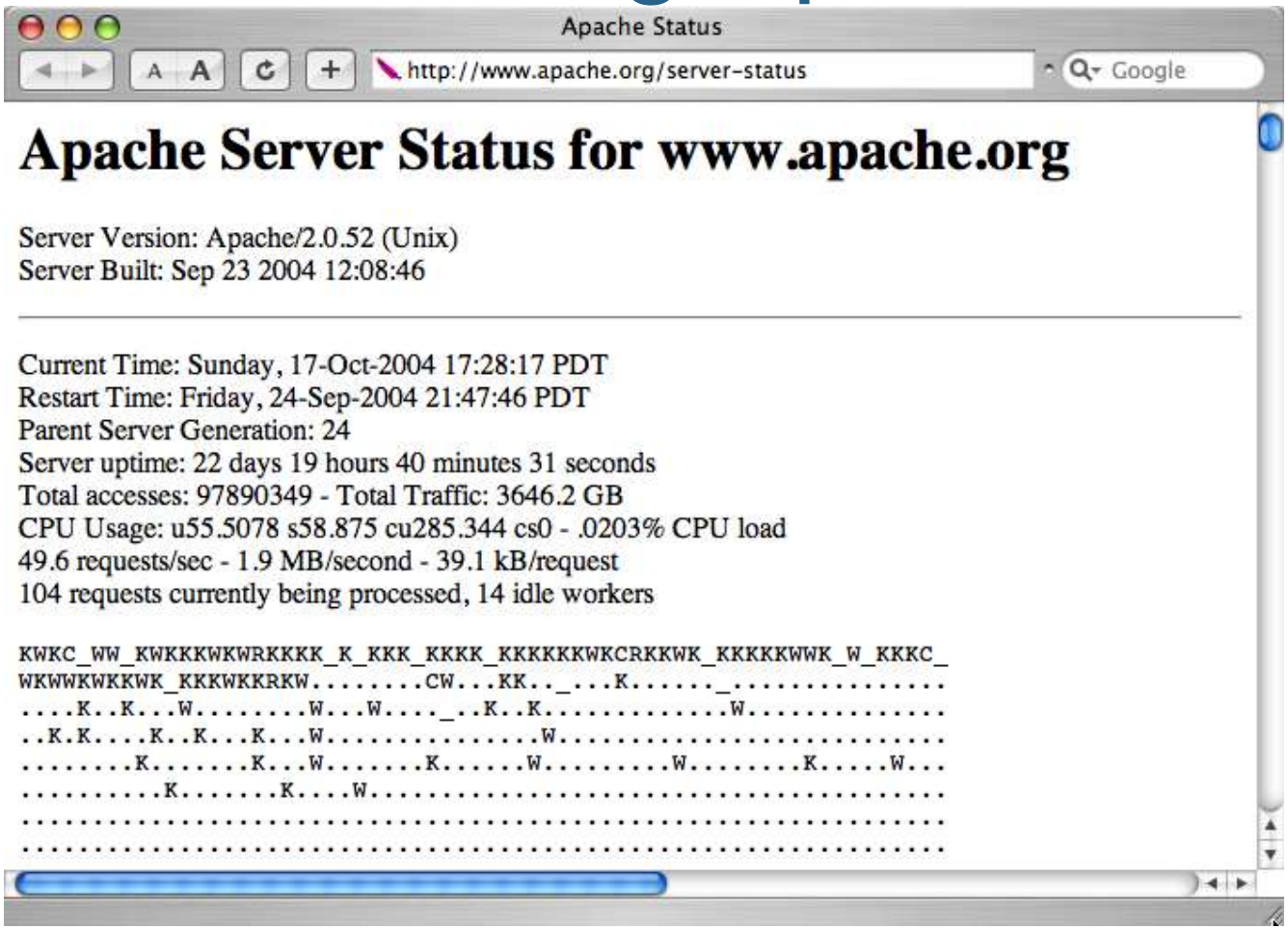
Max In: 11.4 Mb/s (11.4%) Average In: 2755.3 kb/s (2.8%) Current In: 2139.7 kb/s (2.1%)
Max Out: 8678.4 kb/s (8.7%) Average Out: 2856.4 kb/s (2.9%) Current Out: 3002.0 kb/s (3.0%)

Purposes of Monitoring

- Observation
- Extrapolation
- Signals/Alerts
- Testing

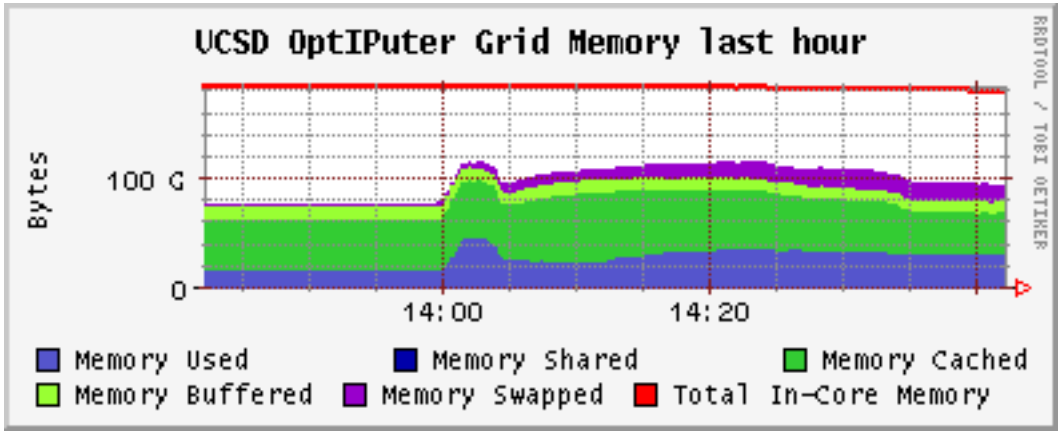


Monitoring Apache



Monitoring: Ganglia

- Long term logging
- Trend Watching
- Free



Other Monitoring Tools

- vmstat
- iostat
- top
- free
- netstat -s



Web Server Logs

- *ErrorLog*
  - LogLevel: debug, info, notice, warn, error, crit
- *Access Log*: TransferLog or CustomLog
  - Common Log Format



Combined Log Format

```
222.127.111.234 - - [09/Apr/2008:07:46:54 +0000] "GET /sander/SanderMugshot2.jpg
HTTP/1.1" 200 22992 "http://people.apache.org/gallery.html" "Mozilla/5.0 (Windows; U;
Windows NT 5.1; en-US; rv:1.8.1.13) Gecko/20080311 Firefox/2.0.0.13"
```

Client IP	222.127.111.234
RFC 1413 ident	-
username	-
timestamp	[09/Apr/2008:07:46:54 +0000]
Request	"GET /sander/SanderMugshot2.jpg HTTP/1.1"
Status Code	200
Content Bytes	22992
Referer	"http://people.apache.org/gallery.html"
User-Agent	"Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.8.1.13) Gecko/20080311 Firefox/2.0.0.13"

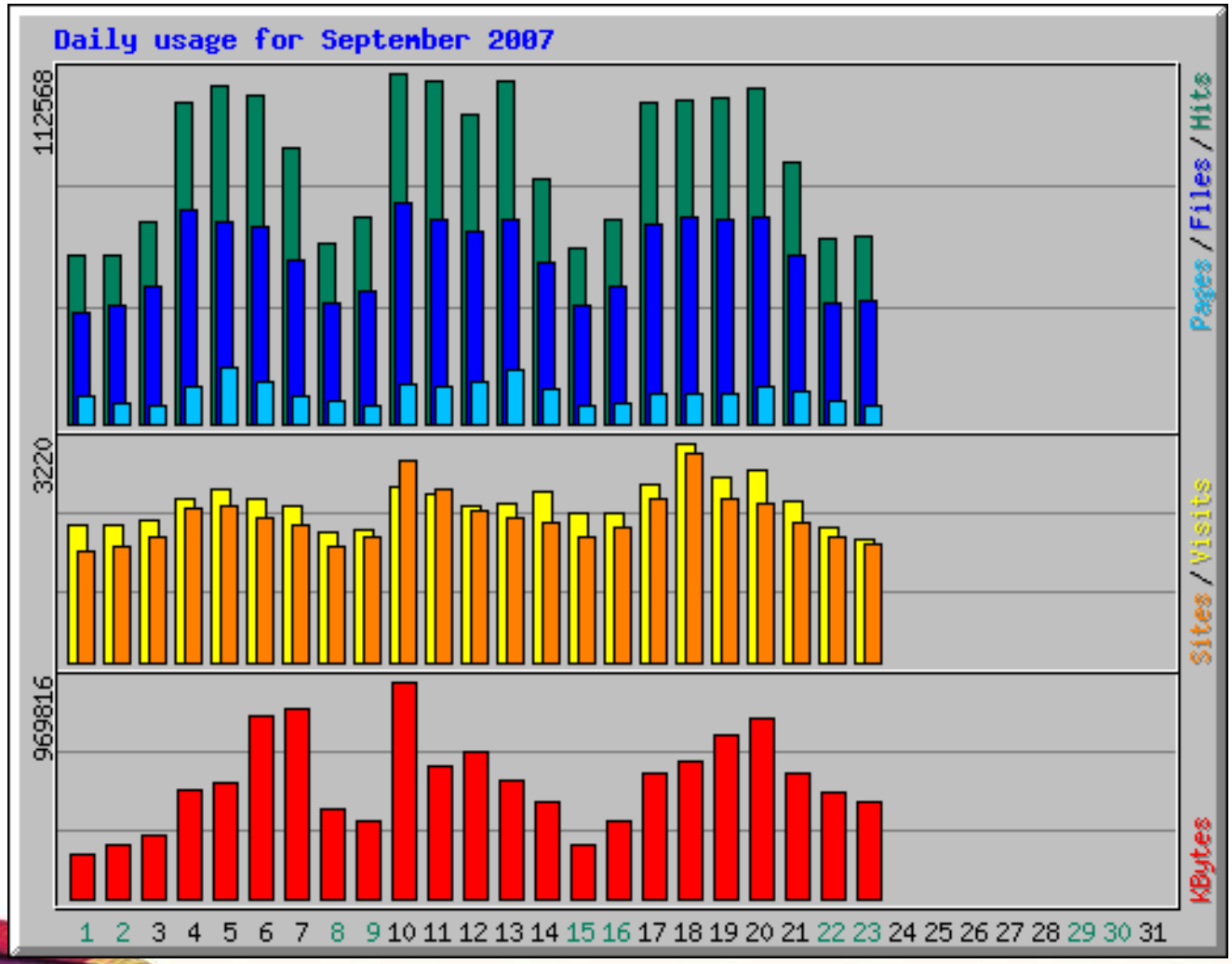
```
"%h %l %u %t \"%r\" %>s %b \"%{Referer}i\" \"%{User-Agent}i\""
```

Log Analysis

- Analyse Apache log files
- Periodically run vmstat, iostat
  - Set up a script
- Watch for Trends
  - Tells you when to buy new kit
- Watch for Peaks
  - And how your system behaves



Log Analysis – Webalizer



Configuring for Performance

- Configuring Apache
- Tuning the Operating System



Apache Configuration

- Process/Thread Management
- DNS Lookups
- Avoid *.htaccess* Files
- Disable unused modules
- Tune your App Tier



DNS Lookups

- HostnameLookups
- Access Control
  - Bad: *Deny from example.com*
  - Good: *Deny from 172.160.234.5*

.htaccess Files

- Per-directory configuration files
- Accessed for every request
- Best performance: *AllowOverride none*

GET /dir1/dir2/restricted.html HTTP/1.0

MaxClients

- Configuration file directive
- Maximum number of workers
- Apache 1.3, 2 Prefork: processes
- Apache 2 Worker: threads * processes
- Limit according to resources (memory, CPU)



Server-side Includes

- We Love Them!
  - Easy to implement
  - Easy to manage
- We Hate Them!
  - They break sendfile()
  - They may break cacheability
- X-bit hack



The X-Bit Hack

- Directive: XBitHack on|off|full
  - Default: off
- On:
  - Every text/html document that has User Execute bit set is parsed
- Full:
  - If Group Execute bit is set, send Last-Modified of file on disk



Sizing MaxClients

- Take total RAM
- Subtract OS allowance
  - look at *free* value without Apache, etc.
- Subtract external program allowance
  - JVM, cgi programs, MySQL?
- Divide by httpd process size
  - Read process size from top



Top

Terminal — bash (ttyp2)

```
 11:37am  up  5:19, 10 users,  load average: 0.84, 0.26, 0.14
184 processes: 181 sleeping, 2 running, 1 zombie, 0 stopped
CPU states: 19.5% user, 13.9% system,  0.0% nice, 66.5% idle
Mem:   635008K av,  612376K used,   22704K free,       0K shrd,  109408K buff
Swap: 1315400K av,      76K used, 1315324K free                   301112K cached

  PID USER     PRI  NI  SIZE  RSS SHARE STAT %CPU %MEM   TIME COMMAND
  850 root      14   0 32424  23M  2972 S     2.7  3.7   3:28 X
    6 root      12   0     0    0     0 SW    0.5  0.0   1:05 kscand
31244 sctemme   10   0  1128 1128   864 R     0.5  0.1   0:00 top
31268 sctemme    9   0  1944 1944  1416 S     0.5  0.3   0:00 httpd
31269 sctemme    9   0  1944 1944  1416 S     0.5  0.3   0:00 httpd
31281 sctemme    9   0  1944 1944  1416 S     0.5  0.3   0:00 httpd
31288 sctemme    9   0  1944 1944  1416 S     0.5  0.3   0:00 httpd
31304 sctemme    9   0  1944 1944  1416 S     0.5  0.3   0:00 httpd
31328 sctemme    9   0  1944 1944  1416 S     0.5  0.3   0:00 httpd
31334 sctemme    9   0  1944 1944  1416 S     0.5  0.3   0:00 httpd
14170 sctemme   10   0 14800  14M 10904 S     0.3  2.3   0:15 kdeinit
31123 sctemme    9   0  1944 1944  1416 S     0.3  0.3   0:00 httpd
31253 sctemme    9   0  1944 1944  1416 S     0.3  0.3   0:00 httpd
31258 sctemme    9   0  1944 1944  1416 S     0.3  0.3   0:00 httpd
31262 sctemme    9   0  1944 1944  1416 S     0.3  0.3   0:00 httpd
31264 sctemme    9   0  1944 1944  1416 S     0.3  0.3   0:00 httpd
31280 sctemme    9   0  1944 1944  1416 S     0.3  0.3   0:00 httpd
```

Selecting Your MPM

- Processes and Threads
- Differences between platforms
- Thread-safety issues



Processes and Threads

- Process:
  - Own copy of data structures
  - Shares: program code, shared memory
  - Context switches expensive
- Thread:
  - Runs within process
  - Shares process environment
  - No context switch



Platforms and Threading

- Context switches expensive on Solaris, AIX
- Context switches cheaper on Linux
- Solaris uses M:N threading
- Linux uses 1 process per thread



Thread-safety

- Third-party modules and libraries
  - mod_perl: experimental threading in Perl 5.6; more mature in Perl 5.8
  - PHP: uses many third-party libraries
- FreeBSD: threading not reliable until 5.x
  - Apache 2.2 Worker MPM runs on FreeBSD

Tune your App Tier

- Tomcat
  - Edit server.xml, tune minProcessors, maxProcessors
  - Use APR -> Persistent connections
  - Tune JVM (Heap, Garbage Collection)
- MySQL
  - Ships with various scenarios in support-files:
    - my-{small,medium,large,huge}.conf
  - PHP & prefork: every child makes a connection
  - Tune `max_connections` variable in my.cnf



System Tuning Tips

- RAM and swap space
- ulimit: files and processes
- Turn off unused services and modules



RAM and Swap

- Swap is disk-based Extension of RAM
- Excessive swapping kills performance
- Tune MaxClients
- Never have more memory than swap
  - Upgrade RAM -> add more swap space



ulimit

- Per-process resource limits
- Built-in command of sh, bash
- Important limits:
  - processes (-u)
  - open files (-n)
- Set in invoking shell
- Code in Apache 2 startup script
  - `ulimit -S -n `ulimit -H -n``
- Linux: /etc/security/limits.conf

SSL Performance

- SSL Processing: a Perfect Storm
  - Crypto is hard
  - Multi-stage handshake
  - Copying and rebuffering
- Effects in many spots
  - CPU load
  - Network latency
  - I/O Contention



SSL Performance Tips

- Add more servers
- Re-use SSL Sessions
  - "Sticky" load balancing
- Have Load Balancer process SSL
  - Plaintext to web servers
- Use accelerator card



Caching Content

- Dynamic Content is Expensive
- Static Content Relatively Cheap
- Several Approaches:
  - Dynamic caching
  - Pre-rendering popular pages (index.rss…)



mod_cache Configuration

```
<IfModule mod_cache.c>
   <IfModule mod_disk_cache.c>

     CacheRoot /raid1/cacheroot
     CacheEnable disk /

     # A page modified 100 min. ago will expire in 10 min.
     CacheLastModifiedFactor .1
     # Always check again after 6 hours
     CacheMaxExpire 21600
   </IfModule>

</IfModule>
```



Make Popular Pages Static

- RSS Feeds
- Popular catalog queries
- … (Check your access log)



Static Page Substitution

```
<Directory "/home/sctemme/inst/blog/httpd/htdocs">

   Options +Indexes

   Order allow,deny
   Allow from all

   RewriteEngine on

   RewriteCond %{REQUEST_FILENAME} !-f
   RewriteCond %{REQUEST_FILENAME} !-d
   RewriteRule ^(.*)$ /cgi-bin/blosxom.cgi/$1 [L,QSA]

 </Directory>
```



Tips

- Observe Before You Act
- Act on Monitoring Results
- Don’t Overload Your System



Q&A



Conference Roadmap

- Apache Web Server Cookbook (Training)
- Break My Site
- Apache Performance Tuning Part 2: Scaling Out



Current Version

http://people.apache.org/~sctemme/ApconEU2008/



Thank You

